Do Spiking Networks Help Flow-Based Intrusion Detection? A Duplicate-Controlled Comparison on CSE-CIC-IDS2018

Özgür Uçar*, Semih Utku*, Pablo Sorrentino†, Amir Javadpour‡ and Fatih Turkmen†
*Dokuz Eylul University, Turkey †University of Groningen, Netherlands
‡MOSA!C Lab / ICTFICIAL Oy, Finland
Corresponding author: ucar.ozgur@outlook.com

***Abstract*—Spiking neural networks (SNNs) are proposed for network intrusion detection because neuromorphic hardware could run them at low power, but most reported results come from random row splits of duplicate-heavy benchmarks without tree baselines. We revisit this on CSE-CIC-IDS2018 (16.2 million flows, 13 classes). Identical feature vectors stay together, the data are split into training, tuning, probability-calibration, alarm-calibration and test partitions, and all models are compared at the same benign false-alarm rate (FAR). Besides a three-branch ensemble SNN we train a parameter-matched multilayer perceptron (MLP), the MLP on a spike-free piecewise-linear encoding, three such MLPs under the SNN's fusion head, a spike-input control that feeds the SNN's codes to a non-spiking network, and tree ensembles. Over five seeds at a 5% FAR, the 4-step SNN reaches a macro-F1 of 0.766 ± 0.011, against 0.755 for the MLP, 0.761 and 0.771 for the single and fused piecewise-linear MLPs, 0.779 for the spike-input control and 0.808 for XGBoost. Only the gap to XGBoost is significant, and the fused piecewise-linear ensemble is not distinguishable from the spike-input control (equivalence not tested), so we find no evidence that spiking dynamics, or spike codes beyond fusion, add accuracy. Four time steps match twelve while cutting synaptic events by a factor of 2.9, yet on published Loihi costs the SNN needs about 3 μJ per flow, 2.5 times the MLP's 1.24 μJ with SRAM weight reads. Like conventional detectors, the frozen models transfer poorly to CIC-IDS2017, detecting 23%–32% of attack flows after recalibration. Code and per-run results are released.**

***Index Terms*—intrusion detection, spiking neural networks, CSE-CIC-IDS2018, data leakage, cross-dataset evaluation, neuromorphic computing**

I. Introduction

Network intrusion detection systems (NIDS) that classify flow records are attractive because flow export is cheap and already deployed in most networks. Learning-based detectors trained on public benchmarks such as CSE-CIC-IDS2018 [1] routinely report accuracies above 99%. Spiking neural networks [2] have joined this line of work, motivated by the low power that neuromorphic chips such as Loihi [3], [4] promise for always-on monitoring.

Two problems make many of these numbers hard to interpret. The CIC datasets contain labelling and feature-extraction errors [5], [6], [7], and a large share of their rows are exact duplicates. When a random row split puts copies of the same vector into both training and test sets, the test score partly measures memorisation [8], [9]. SNN papers also tend to compare against convolutional or recurrent networks trained in a similar way, while gradient-boosted trees, which remain hard to beat on tabular data [10], [11], are left out. Reporting accuracy at the default argmax decision hides the operating point that matters to an analyst, namely how many attacks are caught at a fixed rate of false alarms [12].

This paper asks a narrow question. If the evaluation keeps copies of a training vector out of the test set, calibrates every model the same way and fixes the false-alarm rate, does an ensemble SNN offer an accuracy or efficiency advantage on CSE-CIC-IDS2018? Our contributions are:

- A duplicate-controlled protocol for CSE-CIC-IDS2018, with separate partitions for model selection, probability calibration, alarm calibration and test, applied here to SNN-based detection.
- A spike-input control that feeds the SNN's encodings to a non-spiking network. Together with a parameter-matched MLP and three tree ensembles, it separates the contribution of the encoding from that of the membrane dynamics. A spike-free piecewise-linear MLP, alone and as a three-branch ensemble under the SNN's fusion head, further separates the spike codes from binning and from the fusion architecture.
- An audit of duplicated feature vectors that quantifies how much of the remaining error comes from vectors carrying conflicting labels, extending earlier analyses of labelling errors in this dataset [6].
- A time-window ablation ($T \in \{4, 8, 12\}$) with energy estimates under four cost models, which disagree on whether the SNN is cheaper than the MLP.
- A transfer test of the frozen detectors on CIC-IDS2017 and a label-budget study of local adaptation, which confirm for SNNs the cross-dataset failure reported for other learners [13], [14].

Our results are mixed. The SNN is a competent detector and uses a third of the time steps with no measurable loss in macro-F1. It does not reach the tree ensembles, however, and does not differ significantly from any of the non-spiking networks we trained.

II. Related Work

Dataset quality and cross-dataset tests. Labelling and flow-construction errors are documented for CIC-IDS2017 [5], [15] and CSE-CIC-IDS2018 [6], and Flood et al. [7] catalogue design problems in NIDS benchmarks. Others list evaluation pitfalls [9], [16] and argue for deployment-oriented assessment [12]. Cross-dataset failure is well documented here. D'hooge et al. trained on CSE-CIC-IDS2018 and tested on CIC-IDS2017 and related sets [13], and studied the reverse direction [17]; supervised models rarely learned a representation that carried over. Cantone et al. [14] ran a systematic cross-dataset study over CIC-IDS2017, CSE-CIC-IDS2018 and their corrected versions. Our transfer test repeats this kind of experiment for SNNs.

Tabular baselines and numerical encodings. Tree ensembles still match or beat deep networks on most tabular benchmarks [10], [11]. Gorishniy et al. [18] showed that piecewise-linear and periodic embeddings of scalar features substantially improve plain MLPs. Deterministic spike codes of a scalar are, in effect, fixed binning or radial-basis encodings, so a gain from spike encoding may be an instance of this effect.

Spiking networks and energy. SNNs are trained with surrogate gradients [19], for example in snnTorch [20]. Patel et al. [21] found that latency (spike-timing) coding improved macro-F1 for intrusion detection across their evaluation protocols. Energy claims for SNNs usually rest on an accumulate costing less than a multiply-accumulate [22]. Dampfhoffer et al. [23] showed with a hardware-aware model that includes memory access that SNNs beat efficient ANN implementations only at high spike sparsity. Loihi figures are given in [3], [4].

SNN-based intrusion detection. Zhou and Li [24] took their tree-ensemble figures from earlier work. Wang et al. [25] reported 98.82% accuracy for a convolutional SNN on a six-category sample of CSE-CIC-IDS2018 without a described split, against neural baselines only. Others use NSL-KDD, BoT-IoT or random splits [26], [27], [28]. Patel et al. [21] benchmarked 27 spiking configurations under capture-aware splits, without non-spiking baselines. Hardware evidence comes from a TrueNorth chip [29] and an FPGA [30]. Our contribution is to bring controls that the NIDS evaluation literature already recommends (duplicate-aware splits, tree baselines, fixed false-alarm operating points and cross-dataset tests) to SNN-based detection, and to add a spike-input control. Where Patel et al. ask which code works best, the control asks whether spiking adds anything beyond the code.

Calibration and operating points. We use class-prior correction [31] and temperature scaling [32], and set alarm thresholds in the Neyman–Pearson framing [33]; unlike the umbrella algorithm of Tong et al., our empirical-quantile threshold has no finite-sample guarantee.

III. Data and Protocol

A. Dataset and cleaning

We use the ten daily CSV files of CSE-CIC-IDS2018 from Kaggle (`solarmainframe/ids-intrusion-csv`). After removing repeated header lines and rows with unparseable timestamps, 16,232,929 flows remain. Original labels are mapped to 13 classes; the three web-attack labels (brute force, XSS and SQL injection) are merged into one *Web Attacks* class because together they have only 928 flows. Flow identifiers, IP addresses, source port and timestamp are dropped before learning. Non-finite values are set to zero.

TABLE I
Support per class after the feature-group split. Per-class ratios deviate from 60/10/10/20 because whole duplicate groups go to one partition. Only training is capped.

Class	Train (capped)	Tune	Cal.	Test
Benign	335,515	1,362,310	1,330,511	2,699,900
Bot	16,651	22,981	33,237	54,821
DDoS-HOIC	26,260	68,256	69,218	134,836
DDoS-LOIC-UDP	1,066	159	181	324
DDoS-LOIC-HTTP	23,516	57,929	57,319	115,366
DoS-GoldenEye	10,601	4,135	4,078	8,362
DoS-Hulk	20,762	46,577	45,539	92,601
DoS-SlowHTTPTest	14,486	6,487	568	11,441
DoS-Slowloris	6,422	1,053	1,064	2,451
FTP-BruteForce	16,781	1,879	6,095	7,049
Infiltration	13,497	16,205	16,432	32,470
SSH-BruteForce	13,887	9,435	9,888	61,742
Web Attacks	556	87	106	179

B. Feature-group split

We compute a 128-bit fingerprint of each row's cleaned feature vector and treat all rows with the same fingerprint as one group. Groups, not rows, are assigned to training, tuning, calibration and test partitions in a 60/10/10/20 ratio (split seed 2026). No test vector is seen verbatim in training, but near-duplicates from one attack episode can be, hence "duplicate-controlled". Classes with many duplicates shrink a great deal under this rule. DDoS-HOIC, for example, has 686,012 flows but only 198,861 distinct vectors. Table I gives the resulting support.

The training partition is capped at 500,000 rows. Classes with at most 10,000 training rows are kept in full, and the remaining capacity is shared in proportion to class size. No synthetic oversampling is used. The calibration partition is split again by groups, half for fitting probability calibration and half for choosing alarm thresholds, which gives five disjoint partitions in total.

C. Preprocessing

Every preprocessing step is fitted on the training partition only. Constant columns are removed, columns with absolute skew above 5 get a signed $\log(1 + |x|)$ transform, and a quantile transformer maps each feature to $[0, 1]$. Seventy features remain.

D. Decision rule

All models output 13 class scores z, which we treat as logits (tree ensembles use log-probabilities). Because training was class-capped, we add a prior offset [31], $\tilde{z}_c = z_c + \log \pi_c^{\text{full}} - \log \pi_c^{\text{train}}$. A single temperature T_{cal} is then fitted by minimising negative log-likelihood on up to 100,000 rows

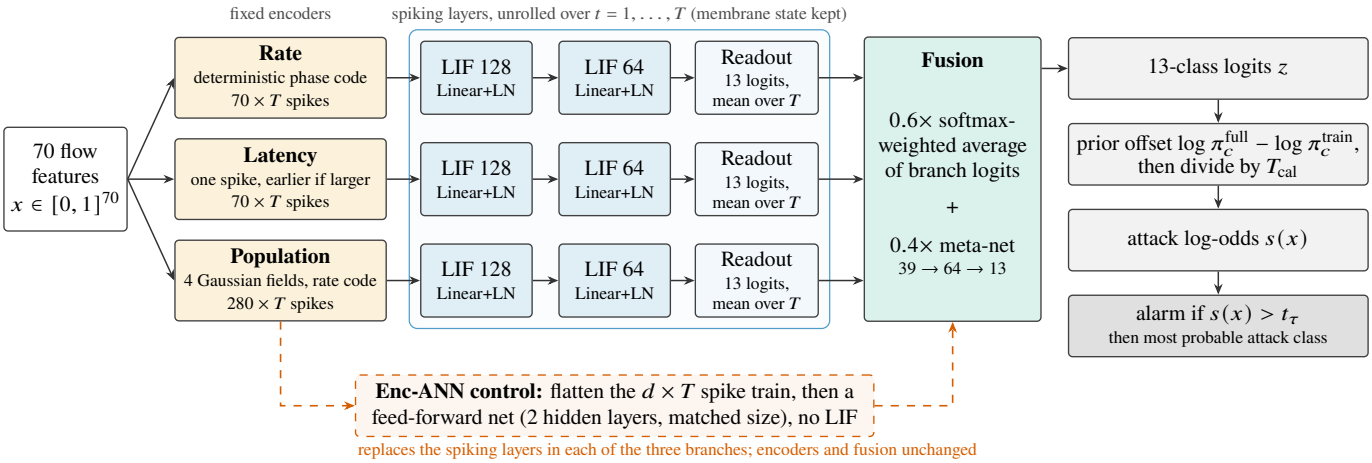


Fig. 1. Ensemble SNN. Three fixed encoders turn the 70 scaled flow features into spike trains over T steps, each branch passes them through two LIF layers and a time-averaged readout, and the branch logits are fused before the prior offset, temperature T_{cal} and alarm threshold t_τ are applied. The time window T and the temperature T_{cal} are unrelated. The dashed path is the spike-input control (Enc-ANN), which replaces the spiking layers with a feed-forward network on the flattened spike train.

of the probability-calibration half [32]. The alarm score is the attack log-odds

$$s(x) = \log \sum_{c \neq \mathrm{B}} e^{\tilde{z}_c / T_{\text{cal}}} - \tilde{z}_{\mathrm{B}} / T_{\text{cal}}, \tag{1}$$

where B denotes the benign class. We call the share of benign flows that raise an alarm the false-alarm rate (FAR) and the share of attack flows that raise an alarm the detection rate (DR). For a FAR budget τ, the threshold t_τ is the empirical $(1-\tau)$ quantile of s over benign rows of the alarm-calibration half. A flow raises an alarm if $s(x) > t_\tau$ and is then assigned the class $\arg\max_{c \neq \mathrm{B}} \tilde{z}_c$. We use $\tau \in \{1\%, 5\%, 10\%\}$ and fixed $\tau = 5\%$ as the primary budget before testing. Test labels are never used to pick a threshold, a temperature or a checkpoint.

IV. Models

Input. Every model receives one static vector of 70 flow statistics per flow, as exported by CICFlowMeter. Besides durations, packet and byte counts, inter-arrival statistics and flag counts, the inputs include the destination port and the protocol number as numeric features; only the source port is dropped. Destination port is a known shortcut in testbed captures, since several attacks target a single service. Retraining without it (three seeds, $T = 4$, same partitions, compared with the three-seed runs that include it) costs one to three points of macro-F1 at the 5% budget: 13-class E-SNN $0.766 \to 0.754$, Enc-ANN $0.787 \to 0.776$, XGBoost $0.808 \to 0.799$; on the 11 identifiable classes 2.0, 2.9 and 1.2 points (XGBoost Holm $p = 0.002$). The consistent losses are on DoS-Slowloris, Web Attacks and Bot, while the two label-conflicted classes move erratically. DR stays within 0.1 points and the ranking is unchanged. We therefore keep it as an input; its role in transfer is examined in Section V-F. No model sees packet sequences; the SNN's time steps only encode each static value as a short spike train.

Ensemble SNN (E-SNN). Three branches (Fig. 1) share the same architecture and differ only in how the 70 features become spikes over $T = 12$ time steps. The *rate* branch uses a deterministic phase-accumulator code: feature x adds x to a phase each step and spikes when the phase crosses an integer, with a fixed golden-ratio offset per feature. The *latency* branch emits a single spike at step $\mathrm{round}((1-x)(T-1))$, so larger values spike earlier. The *population* branch expands each feature into four Gaussian receptive fields with centres evenly spaced on $[0,1]$ and width $1/3$, and rate-codes their activations. Rate coding suits magnitudes such as byte counts, latency coding stresses large values such as long durations, and the population code resolves narrow ranges such as flags, ports and protocols. Each branch has two hidden layers of 128 and 64 leaky integrate-and-fire neurons (decay initialised at 0.9 and learned), layer normalisation before each spiking layer and dropout of 0.2, followed by a linear readout averaged over time. The branch outputs are combined as 0.6 times a softmax-weighted average plus 0.4 times a meta-network ($39 \to 64 \to 13$). This mixture was fixed in an earlier version of the study and not tuned here. The model has 86,013 parameters and is trained with surrogate gradients (fast sigmoid, slope 25) in snnTorch [20].

Matched MLP. A perceptron with two hidden layers of 252 units (layer normalisation, GELU, dropout 0.2), a width chosen by parameter count alone to match the E-SNN (85,945 parameters). It sees the quantile-scaled features directly.

Spike-input control (Enc-ANN). The same three fixed encoders and fusion head as the E-SNN, but each branch flattens the whole spike train and passes it through a two-hidden-layer feed-forward network whose width matches the parameter count of the corresponding SNN branch (84,503 parameters at $T = 12$). As the flattened input grows with T, the hidden width shrinks (21/21/13 units for the rate, latency and population branches at $T = 12$; 53/53/39 at $T = 4$).

Spike-free encoding control (PLE-MLP). A two-hidden-layer MLP on a 16-bin piecewise-linear encoding of each feature [18]. The bins are uniform on the quantile-scaled input, so they are training-set quantile bins. Matching parameters (85,923) on the 1,120-dimensional input leaves 71 hidden units, against 252 in the MLP, so capacity and encoding are confounded. It is a single network without the three-branch fusion of the E-SNN and Enc-ANN.

Fused PLE ensemble. Three PLE-MLP branches under the E-SNN's fusion head, each matched to the mean E-SNN branch size (24 hidden units; 87,183 parameters in total). It keeps Enc-ANN's fusion head but feeds the same piecewise-linear code to all three branches, so code diversity and branch width also differ from Enc-ANN (53/53/39 units at $T = 4$), and part of any gain over the single PLE-MLP may be a plain three-member ensemble effect. At 24 units on a 1,120-dimensional input it is the narrowest model compared, which works against it.

Tree ensembles. Random forest [34] (200 trees, depth 25), XGBoost [35] (400 trees, depth 8, learning rate 0.1, row and column subsampling 0.8, histogram method) and CatBoost [36] (600 iterations, depth 8, learning rate 0.05). These common defaults were not tuned, whereas the neural models were early-stopped on tuning data and the E-SNN went through a variant screen.

Training details. All neural models use AdamW with learning rate 4×10^{-4}, weight decay 10^{-3}, batch size 1,024, gradient-norm clipping at 1.0 and plain cross-entropy, for at most 35 epochs. We keep the checkpoint with the best prior-corrected macro-F1 on the tuning partition and stop after 10 epochs without improvement. The main run trains every model for three seeds (42, 43, 44) at $T = 12$. A second run trains the E-SNN, Enc-ANN, MLP, PLE-MLP and XGBoost at $T = 4$ with five seeds (42–46) and is our primary evidence; its seeds 42–44 reproduce earlier runs exactly, so only seeds 45 and 46 are new. We moved the primary comparison to $T = 4$ and added the PLE-MLP to the declared comparisons after the second review round, when $T = 4$ and $T = 12$ were already known to perform alike on test. A third run, added after the third review round, trains the fused PLE ensemble with the same five seeds on the same partitions. Seeds change initialisation and batch order, not the split. The E-SNN and MLP were also trained with class-weighted cross-entropy. Configurations are in `configs/`.

Follow-up experiments reuse the partitions, preprocessor and decision rule byte for byte and refuse to run if data hashes differ. They also retrain the E-SNN and Enc-ANN at $T = 4$ and 8 (three seeds) and screen five E-SNN variants.

V. Results

A. Detection at a fixed FAR budget

The achieved test FAR is 4.9% to 5.0% for every model (Table II), and the KS distance between benign calibration and test scores is 0.005 to 0.014.

All models catch about 96% of attack flows, and DR spans only 0.9 points. More than 99% of the missed attack flows are Infiltration, so DR says little about the models. The separation shows up in macro-F1, which weights rare and difficult classes equally. We test four declared comparisons for the E-SNN on the five-seed $T = 4$ run (Welch, Holm within each metric and budget; the XGBoost gap at 5% also survives Holm over all 16 tests). At the 5% budget the E-SNN differs from the matched MLP by $+1.1$ points (95% CI $[-0.3, 2.5]$, Holm $p = 0.32$), from the PLE-MLP by $+0.5$ ($[-0.9, 1.9]$, $p = 0.51$), from Enc-ANN by -1.3 ($[-3.8, 1.2]$, $p = 0.51$) and from XGBoost by -4.2 ($[-5.6, -2.8]$, $p = 0.002$). At 1% only the XGBoost gap is significant (-5.2, $[-7.3, -3.1]$, $p = 0.007$), as on the 11 identifiable classes (-3.0 at 5%, $p = 0.031$; -4.2 at 1%, $p = 0.041$). With five seeds a Welch test has 80% power for gaps of about 2 points against the MLP and PLE-MLP and 3.4 points against Enc-ANN. The three-seed $T = 12$ run gives the same picture (Holm $p \geq 0.25$ at 5%). These intervals cover seed variance; the split seed is fixed.

TABLE II
Test results at a 5% FAR budget (mean ± s.d.). Top: $T = 4$, five seeds (primary). Bottom: $T = 12$, three seeds. Trees do not depend on T; the top XGBoost row adds seeds 45–46. MF1-11 excludes FTP-BruteForce and DoS-SlowHTTPTest (Section V-B). DR and FAR are seed means.

Model	Macro-F1	MF1-11	DR	FAR
$T = 4$, *five seeds*				
E-SNN	0.766 ± 0.011	0.828 ± 0.014	0.959	0.049
Matched MLP	0.755 ± 0.004	0.820 ± 0.004	0.958	0.050
PLE-MLP	0.761 ± 0.005	0.824 ± 0.003	0.959	0.049
PLE ensemble	0.771 ± 0.013	0.834 ± 0.016	0.959	0.049
Enc-ANN	0.779 ± 0.020	0.842 ± 0.013	0.961	0.049
XGBoost	0.808 ± 0.004	0.858 ± 0.001	0.967	0.049
$T = 12$, *three seeds*				
E-SNN	0.769 ± 0.022	0.837 ± 0.019	0.959	0.049
Matched MLP	0.756 ± 0.004	0.821 ± 0.005	0.958	0.050
Enc-ANN	0.787 ± 0.017	0.848 ± 0.017	0.963	0.049
Random forest	0.788 ± 0.001	0.842 ± 0.001	0.965	0.050
CatBoost	0.791 ± 0.005	0.843 ± 0.004	0.965	0.049
XGBoost	0.808 ± 0.005	0.858 ± 0.001	0.967	0.049

For equivalence we use a 2-point margin, the five-seed MDE80, chosen in the third review round after the data were seen. TOST shows the E-SNN equivalent to the PLE-MLP at the 5% budget ($p = 0.020$; 11 classes $p = 0.031$), but not at 1% ($p = 0.087$) and not to the MLP ($p = 0.074$); the PLE-MLP is itself equivalent to the MLP at 5% ($p = 0.001$), in line with its 71-unit capacity limit. In exploratory comparisons Enc-ANN is 2.4 and 3.7 points above the MLP at 5% and 1% (90% CIs exclude zero) and 1.8 to 2.2 above the PLE-MLP. The fused PLE ensemble reaches 0.771 ± 0.013 (0.794 at 1%), 1.0 points above the PLE-MLP and 0.8 below Enc-ANN (95% CI $[-3.4, 1.7]$). None of its differences from the neural models is significant (against the MLP $+1.5$, $p = 0.059$), and it trails XGBoost by 3.7 ($p = 0.002$; all unadjusted). The point estimates are consistent with fusion/ensembling accounting for about half of Enc-ANN's lead over the PLE-MLP, but neither step is individually significant (ensemble minus PLE-MLP $+1.0$, $p = 0.19$). Neither spiking dynamics nor, beyond fusion/ensembling, the spike codes add accuracy

TABLE III

PER-CLASS TEST F1 AT A 5% FAR BUDGET, $T = 4$, MEAN OF FIVE SEEDS. THE EIGHT CLASSES WITH F1 ≥ 0.96 FOR EVERY MODEL ARE OMITTED (DOS-SLOWLORIS RANGES FROM 0.977 FOR THE E-SNN TO 0.998 FOR XGBOOST). RANDOM FOREST AND CATBOOST EXIST ONLY IN THE THREE-SEED $T = 12$ RUN.

Class	E-SNN	MLP	PLE	PLE ens.	Enc-ANN	XGB
DDoS-LOIC-UDP	0.816	0.823	0.815	0.818	0.823	0.838
DoS-SlowHTTPTest	0.505	0.521	0.507	0.506	0.491	0.728
FTP-BruteForce	0.341	0.273	0.324	0.341	0.372	0.332
Infiltration	0.131	0.126	0.130	0.132	0.141	0.172
Web Attacks	0.261	0.151	0.174	0.277	0.337	0.465

that we can detect. An approximate bootstrap over the 2.29 million test fingerprint groups (200 replicates) holds models and thresholds fixed, so it excludes threshold-refit variance. Its intervals are narrow (E-SNN DR 95.89% [95.47, 96.46] at 5%), and every paired DR and AUC difference keeps its sign (E-SNN minus XGBoost: -0.84 DR points, $[-0.93, -0.72]$). These binary gaps stay below one point while the macro-F1 gaps to XGBoost and Enc-ANN are several points, so those models differ mainly in which attack class they assign, not in whether they alarm. Macro-F1 is not bootstrapped because per-row class predictions were not saved.

In the three-seed run the order at 1% matches 5%; at 10% Enc-ANN (0.767) passes random forest (0.763). The prior offset carries the whole calibration effect (temperatures 0.96 to 1.01), and class-weighted training did not help (E-SNN 0.753, MLP 0.747).

Per capture day the FAR reaches 24.1% (E-SNN) and 32.3% (XGBoost) on the Infiltration days, plausibly from unlabelled malicious traffic [6].

B. Where the errors are

Table III lists per-class F1 for the five-seed run. Eight of the thirteen classes are detected almost perfectly (F1 ≥ 0.96) by every model. The differences between models come from the other five: DDoS-LOIC-UDP, DoS-SlowHTTPTest, FTP-BruteForce, Infiltration and Web Attacks. The neural ranking rests largely on Web Attacks, with only 179 test flows (F1 0.151 for the MLP, 0.174 for the PLE-MLP, 0.277 for the PLE ensemble and 0.337 for Enc-ANN), and on the unidentifiable FTP-BruteForce class (0.273 to 0.372).

The feature-group audit explains part of this. All 193,360 FTP-BruteForce flows fall into just 54 distinct vectors, and 193,353 of them lie in groups that also carry another label. For DoS-SlowHTTPTest, 139,884 of 139,890 flows share vectors with other classes, and the three largest groups in the dataset mix these two classes. Both attacks leave short, nearly empty TCP connections that the extractor summarises identically, consistent with Liu et al. [6]. No classifier can separate such flows, so their test F1 mostly reflects how a few large groups fell in the split. Removing the two classes (MF1-11 in Table II) narrows the gap between the E-SNN and random forest at $T = 12$ from 1.9 to 0.5 points and puts Enc-ANN above random forest and CatBoost (at $T = 4$ it ties random forest at 0.842), so much of the 13-class ranking rests on these two classes. Infiltration is spread over 140,610 groups, and 88,810 of its rows lie in groups that also carry another label.

Overall, 68,467 groups carry conflicting labels, and predicting each group's majority label would still be wrong on 1.10% of rows. Since identical inputs cannot be separated even by memorisation, a lower error on these features implies a different label set or preprocessing, not leakage, and is not comparable with ours.

Web Attacks has high ROC AUC (0.994 for the seed-42 E-SNN at $T = 12$, 1.000 for XGBoost) but low F1, because its 179 test flows face 2.7 million benign ones: the E-SNN finds 98 of them with 661 false assignments, XGBoost 160 with 342. Average precision (0.49 and 0.85) reflects this better than AUC.

TABLE IV

OPERATIONS AND ESTIMATED ENERGY PER FLOW (μJ), SEED 42, TEST CLASS MIX. 45 NM INCLUDES LAYER NORMALISATION AND MEMBRANE UPDATES [22]; +SRAM ADDS ONE SRAM READ PER WEIGHT USE; INT8 IGNORES THE WIDER MEMBRANE ACCUMULATOR; LOIHI USES PUBLISHED COSTS [3].

Model	T	ACs	MACs	45 nm	+SRAM	INT8	Loihi
E-SNN	4	120,028	3,367	0.18	1.41	0.007	3.1
E-SNN	8	233,577	3,367	0.33	2.70	0.013	6.0
E-SNN	12	347,762	3,367	0.49	4.00	0.019	9.0
Enc-ANN	4	31,349	12,352	0.09	0.53	0.004	–
Enc-ANN	8	31,034	6,568	0.06	0.44	0.003	–
Enc-ANN	12	30,153	5,094	0.05	0.41	0.002	–
MLP	–	0	84,420	0.40	1.24	0.020	–

C. Operation counts and energy

We counted operations with forward hooks on flows from four capture days and weighted the per-class counts by test-partition support. The eight classes measured cover 92% of test rows. A linear layer with binary input costs one accumulate (AC) per nonzero input and output neuron; other layers cost multiply-accumulates (MACs).

At $T = 12$ the E-SNN's hidden neurons are silent 72% of the time, yet with 12 steps and three branches it still performs four times as many synaptic events as the MLP performs MACs. With 45 nm costs (0.9 pJ per AC, 4.6 pJ per MAC) the E-SNN's linear layers alone cost 0.33 μJ at $T = 12$, below the MLP's linear layers (0.39 μJ). Its layer normalisation and membrane updates run at every time step, though, and with them the E-SNN costs 0.49 μJ, about 23% above the MLP's 0.40 μJ; at $T = 4$ it stays 55% below. With SRAM weight reads the E-SNN costs more than the MLP at every T (1.41 against 1.24 μJ at $T = 4$). Enc-ANN is far cheaper than both because it processes each binary input once instead of propagating it through membrane state. On one CPU thread the E-SNN is 40 to 50 times slower than the MLP (57 against 1.4 μs per flow, and 105 against 2.1 in a repeat).

On Loihi each synaptic event (a spike times its fan-out) costs 23.6 pJ and each neuron update 81 pJ [3]. The E-SNN then needs 3.1 to 9.0 μJ per flow, 2.5 to 7.2 times the memory-

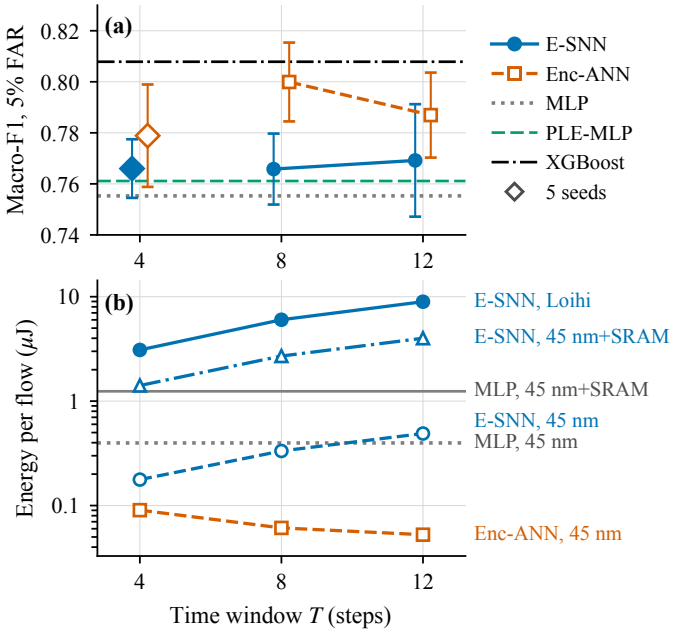


Fig. 2. Effect of the time window T. (a) Test macro-F1 at a 5% FAR budget (mean and s.d.; diamonds: five seeds at $T = 4$, unconnected to the three-seed points at $T = 8$ and 12), with the five-seed MLP, PLE-MLP and XGBoost as references. (b) Estimated energy per flow on a log scale, as in Table IV: complete 45 nm model, with SRAM reads, and Loihi costs for the E-SNN.

inclusive MLP cost with SRAM weight reads, which Loihi's per-event figure also includes (7.8 to 22.5 times the arithmetic-only 45 nm cost). Under INT8 arithmetic the E-SNN is cheaper than the MLP at $T = 4$ and 8 and at parity at $T = 12$ (0.019 against 0.020 μJ). Synaptic events, three quarters from the input encoding, make up over 90% of the E-SNN total; input injection, static power and routing are not costed.

D. Shorter time windows

Fig. 2 repeats the neural comparison with 4 and 8 time steps on the same partitions. For the E-SNN, three-seed macro-F1 at the 5% budget is 0.766, 0.766 and 0.769 at $T = 4$, 8 and 12 (0.791, 0.800, 0.789 at 1%), with no detectable difference; Enc-ANN reaches 0.787, 0.800 and 0.787. With hidden sparsity of 72% to 74%, synaptic events fall 2.9-fold from $T = 12$ to 4. The 90% CI of the $T = 4$ minus $T = 12$ difference at 5% is $[-3.7, 3.1]$ points, so a 2-point effect is not excluded.

E. Architecture screen

Five E-SNN variants ($T = 8$, seed 42; wider layers, weighted-average fusion, higher learning rate, eight receptive fields, population branch only) were ranked on tuning macro-F1. On test they reached 0.753–0.768 against 0.756 for the original, within its seed spread; the top-ranked wide variant, retrained with three seeds, reaches 0.770 ± 0.014 against 0.766 ± 0.014 with 2.5 times the parameters.

F. Transfer to CIC-IDS2017

We ran the frozen seed-42 models, unchanged, on the CIC-IDS2017 MachineLearningCVE files from the Kaggle dataset `sweety18/cicids2017-full-dataset` (2,214,468 labelled flows). It was recorded a year earlier on another testbed with an older flow extractor. Our copy lacks the Tuesday and Thursday-morning files, so brute-force and web attacks are absent; the corrected releases [5], [15] would allow a fuller test. Columns are mapped by name (the map is released with the records); the protocol field is missing and is imputed from the destination port. Attack labels are mapped to the 2018 families; PortScan and Heartbleed have no 2018 counterpart. Against benign flows from four 2018 days, the median KS distance over the 70 inputs is 0.245. The largest shifts point to the extractor version: Fwd Seg Size Min (0.485), RST and ECE flags that are zero in 99.97% of 2017 but 68% of 2018 benign flows, and an initial backward window with median -1 against 149. Only 7 of the 70 inputs shift more across the datasets than between the two most different 2018 days, so network and extractor effects are mixed in the failure below.

TABLE V
FROZEN 2018 MODELS ON CIC-IDS2017. "RECAL.": THRESHOLD REFITTED ON HALF OF THE 2017 BENIGN FLOWS (5% BUDGET), ATTACK LABELS UNUSED.

Model	2018 threshold FAR	2018 threshold DR	Recal. FAR	Recal. DR
E-SNN ($T = 12$)	0.262	0.630	0.050	0.230
E-SNN ($T = 4$)	0.228	0.420	0.050	0.239
XGBoost	0.252	0.494	0.050	0.317

Table V confirms for SNNs the transfer failure reported for conventional learners on this dataset pair [13], [17], [14]. With the 2018 threshold the FAR rises from 5% to 23%–26%. Refitting each threshold on half of the 2017 benign flows, the one adjustment possible without attack labels, leaves a DR of 23%–32% on the other half and all attacks. For the SNNs only the slow application-layer DoS attacks appear to transfer: the 12-step E-SNN still flags 86% of GoldenEye and 87% of Slowloris flows, and the 4-step E-SNN 99.8% and 99.6%, while XGBoost flags only 28.8% of GoldenEye. This depends on the destination port. Retrained without it, the 4-step E-SNN flags 32.5% of GoldenEye and 68.8% of Slowloris flows. Removing the port does not repair transfer either: at the 2018 threshold the FAR is still 18% (E-SNN) and 22% (XGBoost), and after recalibration DR is 13.5% and 31.0%, against 23.9% and 31.7% with the port. The port-free Enc-ANN transfers best of all models tested (FAR 12.7% at the 2018 threshold, recalibrated DR 34.4%, AUC 0.686), which is still poor. Fig. 3 gives the recalibrated DR by attack type; no model detects Infiltration or Heartbleed.

To estimate the local labels an operator would need, we split the CIC-IDS2017 flows by feature-vector fingerprint into an adaptation pool, a calibration partition and a test partition (50/10/40) and drew class-stratified labelled samples of 0.1%, 1% and 5% of the pool (1,101, 11,026 and 55,153 flows). We call this the label budget; PortScan was added as a new class. The 5% alarm threshold was refitted on about 83,000 known-benign flows of the 2017 calibration partition. Table VI fits

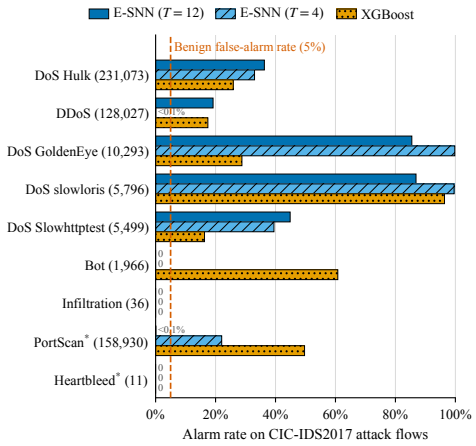


Fig. 3. Share of CIC-IDS2017 attack flows that raise an alarm after the threshold of each frozen 2018 model is refitted to a 5% benign FAR (dashed line). Flow counts are in parentheses; an asterisk marks attack types with no CSE-CIC-IDS2018 counterpart.

TABLE VI
LOCAL ADAPTATION ON CIC-IDS2017 (TEST PARTITION, MEAN OVER THREE SEEDS) BY LABEL BUDGET, THE LABELLED SHARE OF THE ADAPTATION POOL. BUDGET 0 IS THE FROZEN 2018 MODEL. ALL ROWS USE A 5% FAR THRESHOLD REFITTED ON 2017 CALIBRATION DATA.

Method		0	0.1%	1%	5%
E-SNN T=4, fine-tuned	FAR	0.062	0.052	0.059	0.053
	DR	0.268	0.985	0.994	0.996
	MF1	0.188	0.564	0.592	0.707
XGBoost, from scratch	FAR	0.050	0.050	0.050	0.051
	DR	0.314	0.996	1.000	1.000
	MF1	0.291	0.526	0.665	0.686
XGBoost, continued	FAR	–	0.051	0.050	0.057
	DR	–	0.723	0.724	0.749
	MF1	–	0.510	0.494	0.483

the temperature on up to 100,000 labelled calibration flows; fitting it on the label budget alone, or dropping it, changes DR and FAR by at most 0.001, so the label count is the budget itself. Its frozen E-SNN runs at a 6.2% FAR; at an exact 5% test FAR it detects 24.8%, close to the 23.9% of Table V.

With 1,101 labelled flows, fine-tuning the 4-step E-SNN from its 2018 checkpoint raises DR to 98.5% at a 5.2% FAR, and XGBoost trained from scratch on the same flows reaches 99.6% at 5.0% without any pre-training. At an equal test FAR of 1% the gap widens to 95.9% against 99.0%. The fine-tuned E-SNN matches XGBoost on DoS, DDoS and PortScan traffic but not on Bot flows, which it misses with 0.1% and 1% of the pool and detects at 32% with 5%; XGBoost detects 97% to 99.5% of Bot flows with 1% or more. Continuing the 2018 XGBoost model cannot learn PortScan and stays at 72% to 75% DR. Macro-F1 stays between 0.48 and 0.71 because a 5% FAR on 671,643 benign test flows produces about 33,000 false alarms, which swamp the precision of the small classes. Pool and test share capture days and attack episodes, so this same-capture re-split is an optimistic bound; it shows that the data become easy once in-domain labels exist, not an SNN-specific effect.

VI. DISCUSSION AND LIMITATIONS

Effect of spiking dynamics. Membrane dynamics add no accuracy that this protocol can detect. The point estimates are consistent with the architecture around the encoder mattering more than the code: piecewise-linear binning [18] under fusion/ensembling recovers about half of Enc-ANN's lead over a single PLE-MLP, but none of these steps is individually significant. Enc-ANN has the highest mean macro-F1 among the neural models, though not significantly above the E-SNN (Holm $p = 0.51$), and it is cheaper under every cost model in Table IV that applies to it. SNN advantages with on-chip encoding or on per-packet sequences, where time steps are real time, remain possible and are the natural next experiment. For SNN papers on flow features, a spike-input control is the comparison that would show whether membrane dynamics contribute.

Energy depends on the cost model. A 45 nm adder cost favours the SNN at short windows when memory traffic is ignored; with SRAM reads, or with the published Loihi cost of a synaptic event, the MLP is cheaper at every T (the E-SNN on Loihi costs 2.5 to 7.2 times the MLP with SRAM reads), mainly because the input encoding fans out to 128 neurons. This is an intrusion-detection instance of the sparsity threshold reported by Dampfhoffer et al. [23]. We therefore give the cost model with every energy figure.

Tree ensembles. Untuned XGBoost had the highest mean at every budget, in line with evidence on tabular data [10], [11], which makes it the baseline a neuromorphic detector has to meet.

Limitations. The protocol was designed after test results of an earlier row-split version were seen, and later experiments followed the main test results. These are development results on one reused test partition, so we report every configuration we ran; a fresh held-out dataset is still needed. Feature groups prevent verbatim duplication but not near-duplicates or episode-level leakage within an attack run, and the dataset's timestamps are unverified, so we did not attempt a time-blocked split. Destination port is an input and acts as a shortcut worth one to three macro-F1 points. The five-seed run reuses the test partition, its primary role and the PLE-MLP and PLE-ensemble comparisons were fixed after review, and only two of its seeds are new. We used three or five seeds but a single split seed, so no interval covers a redrawn split; the group bootstrap covers test sampling for DR, FAR and AUC only. Tree hyperparameters were fixed and only five SNN variants were screened. The CIC-IDS2017 copy is incomplete, and all energy figures are estimates, not hardware measurements.

VII. CONCLUSION

On CSE-CIC-IDS2018 under a duplicate-controlled protocol with matched false-alarm rates, an ensemble SNN detects 96% of attack flows at a 5% FAR. Across five seeds it does not differ significantly from a parameter-matched MLP, from piecewise-linear MLPs with or without fusion/ensembling,

or from a non-spiking network fed its spike codes, while XGBoost is about 4 macro-F1 points ahead (Holm $p = 0.002$). Four time steps instead of 12 cut its synaptic events 2.9-fold at no measurable cost, yet on published Loihi figures it still needs more energy per flow than the MLP. Much of the remaining error comes from identical vectors with different labels, and no model keeps its accuracy on CIC-IDS2017 without local labels.

Reproducibility

Code, configuration, records and seed-42 checkpoints (E-SNN at $T{=}4$ and 12, Enc-ANN at $T{=}12$, MLP, XGBoost) are at https://github.com/ozgurucar/snn-ids2018 (commit tagged `v1.4`). Other checkpoints and the per-row test scores used for the bootstrap are available from the authors on request. Package versions are pinned in the records.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[3] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[4] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. Fonseca Guerra, P. Joshi, P. Plank, and S. R. Risbud, "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proc. IEEE*, vol. 109, no. 5, pp. 911–934, 2021.

[5] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[6] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 254–262.

[7] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 658–675.

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2022.

[11] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[12] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," in *IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, 2023, pp. 592–614.

[13] L. D'hooge, M. Verkerken, T. Wauters, F. De Turck, and B. Volckaert, "Investigating generalized performance of data-constrained supervised machine learning models on novel, related samples in intrusion detection," *Sensors*, vol. 23, no. 4, p. 1846, 2023.

[14] M. Cantone, C. Marrocco, and A. Bria, "Machine learning in network intrusion detection: A cross-dataset generalization study," *IEEE Access*, vol. 12, pp. 144 489–144 508, 2024.

[15] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, ser. LNCS, vol. 13857. Springer, 2023, pp. 18–33.

[16] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[17] L. D'hooge, M. Verkerken, T. Wauters, F. De Turck, and B. Volckaert, "Characterizing the impact of data-damaged models on generalization strength in intrusion detection," *J. Cybersecurity and Privacy*, vol. 3, no. 2, pp. 118–144, 2023.

[18] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2203.05556.

[19] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[20] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[21] R. Patel, S. Mitra, D. Amebley, T. Akinrele, S. Dibbo, S. Mittal, and S. Rahimi, "The value of spike timing: A leakage-resistant benchmark of SNN design choices for network intrusion detection," in *IEEE Int. Conf. Cognitive Machine Intelligence (CogMI)*, 2026, to appear. arXiv:2606.01442.

[22] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2014, pp. 10–14.

[23] M. Dampfhoffer, T. Mesquida, A. Valentian, and L. Anghel, "Are SNNs really more energy-efficient than ANNs? An in-depth hardware-aware study," *IEEE Trans. Emerging Topics in Computational Intelligence*, vol. 7, no. 3, pp. 731–741, 2023.

[24] S. Zhou and X. Li, "Spiking neural networks with single-spike temporal-coded neurons for network intrusion detection," in *Proc. 25th Int. Conf. Pattern Recognition (ICPR)*, 2021, pp. 8148–8155, arXiv:2010.07803.

[25] Z. Wang, F. A. Ghaleb, A. Zainal, M. M. Siraj, and X. Lu, "An efficient intrusion detection model based on convolutional spiking neural network," *Scientific Reports*, vol. 14, p. 7054, 2024.

[26] M. Živadinović and D. Simić, "Resource efficient Internet-of-Things intrusion detection with spiking neural networks," in *Position Papers of the 19th Conf. Computer Science and Intelligence Systems (FedCSIS)*, ser. Annals of Computer Science and Information Systems, vol. 40, 2024, pp. 73–78.

[27] A. Fesu, N. Shone, Á. MacDermott, B. Zhou, and M. H. Eiza, "Comparative analysis of SNN and CNN models for energy efficient intrusion detection," in *Proc. 12th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2025, pp. 1–7.

[28] M. G. Karthik, V. Keerthika, S. V. Mantena, D. Siri, L. P. Yeluri, K. K. Lella, and B. R. Ganesh, "Energy-efficient intrusion detection with a protocol-aware transformer–spiking hybrid model," *Scientific Reports*, vol. 16, p. 7095, 2026.

[29] M. Z. Alom and T. M. Taha, "Network intrusion detection for cyber security on neuromorphic computing system," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2017, pp. 3830–3837.

[30] A. Mehrabi and A. van Schaik, "Intrusion detection on industrial networks using neuromorphic computing," in *Proc. 6th Int. Conf. Electrical, Control and Instrumentation Engineering (ICECIE)*, 2024, pp. 1–8.

[31] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.

[32] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[33] X. Tong, Y. Feng, and J. J. Li, "Neyman–Pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, p. eaao1659, 2018.

[34] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[35] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[36] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.